BrewFS: Delta-Based Multi-Agent Workspaces with Packed Metadata and Variable-Granularity Object Storage

Internal research draft
26 October 2026

Abstract

Coding agents can share a repository baseline while keeping edits private, but sharing does not determine metadata and data access costs. BrewFS separates shared-version identity from metadata and object access units through three independently optional mechanisms. Delta-Based Workspaces resolve a shared baseline with private changes; Packed Metadata places names, attributes, and data locators in indexed object-resident pages; Variable-Granularity Object Storage separates client object formation from optional backend encoding and verification. Our hypothesis is that these choices reduce lifecycle storage work when reuse or selective access repays construction and maintenance, under matched visibility and durability contracts. We explain inode identity, atomic namespace updates, truncation, publication, and persisted layouts, then identify the implementation boundaries relevant to their evaluation. A reanalysis of 1,138,884 FUSE requests from eight coding-agent task runs characterizes metadata traffic, read/write sizes, and temporal heterogeneity. The planned evaluation isolates overlay, metadata offload, and object granularity, then tests fork, migration, and concurrent workloads with JuiceFS as the external comparator. It accounts for construction, retention, caching, and failure outcomes. The workload observations do not establish system speedups.

Status. This is an author working document: design, source audit, and measured passthrough workload observations are distinguished from planned system experiments. Integration, correctness, and performance claims require the evidence described below.

1 Introduction

Coding agents work through ordinary file-system tools: they traverse a repository, inspect source files, edit code, and run builds and tests [19, 22]. Consider a service that launches several attempts from the same repository revision and dependency set. Each attempt needs a consistent starting view and independent changes. Serving it requires both provisioning that view and resolving names, attributes, and file data throughout execution.

Sharing a baseline addresses provisioning, but leaves these access costs open. Layered namespaces and copy-on-write branches already let tasks reuse state while keeping changes private [12,18,23]. Immutable distributions provide indexed, demand-loaded state [2,14]. A mature immutable lower with a compatible writable upper and object backend is therefore a strong direct alternative. Any comparison with this composition must include its sharing and caching. The remaining question is how the metadata and object units fetched by a task affect request count, transferred bytes, and processing work.

We reanalyzed 1,138,884 FUSE requests from eight coding-agent task runs (Section 2.3). Metadata operations, including unsuccessful probes, made up more than half of the requests in each of the two workload groups. Reads and writes had mixed request lengths, and activity varied substantially across tasks and over time. These observations motivate examining access costs alongside provisioning; request counts alone do not identify a latency bottleneck or establish cross-task reuse.

Our design separates the state a task shares from the units used to access that state. A workspace resolves private changes before querying its pinned lower version; lower answers need not depend on one metadata representation. Data locators connect those answers to object ranges, whose interpretation must preserve the referenced bytes across storage layouts. These interfaces allow different access units to serve the same logical file view. Our hypothesis is that separating shared-version identity from metadata and object access units can reduce lifecycle storage work under matched visibility and durability contracts. The expected savings depend on unchanged-state reuse, metadata locality, or selective data access, and must exceed construction and maintenance costs. Direct composition may already capture those savings; matching or exceeding its cost would weaken the case for the corresponding mechanism.

BrewFS realizes this design in an object-backed file system with three mechanisms. *Delta-Based Workspaces* resolve a shared immutable baseline with private namespace and data-mapping changes. *Packed Metadata* groups immutable names, attributes, and data locators into indexed pages on object storage, moving fixed-version queries away from the central metadata database while mutable and control records remain there. *Variable-Granularity Object Storage* separates client object formation from optional backend encoding and verification units. Each addresses a different cost and remains optional: workspaces can use native metadata, packed snapshots can serve read-only mounts, and ordinary volumes can configure object granularity. Section 3 defines their composition interfaces.

The paper develops private-view resolution, immutable metadata lookup and publication, and file-to-object access rules. It connects those rules to the inspected implementation (Section 7) and the completed request-characterization study. System performance and failure experiments remain planned (Section 8); neither the observations nor source inspection establishes a system speedup.

2 Background and Motivation

2.1 Task views and reusable state

A coding-agent workspace contains source files, dependencies, generated artifacts, and temporary files. Two attempts starting from the same revision may initially read the same `src/parser.py`. If one edits or deletes that file, the other must continue to see its own version. The shared starting state must therefore have a stable identity, and each attempt's changes must remain private, regardless of where metadata and bytes reside.

Reuse is conditional on the deployment. A stable dependency tree may serve many attempts, whereas generated outputs may be replaced on every run. A baseline can cover a selected subtree; it need not include the entire working directory. The value of sharing depends on overlap between tasks, the amount of private modification, and how long the baseline remains useful. A single task's request distribution cannot establish those cross-task properties.

2.2 From file operations to storage work

To read `src/parser.py`, a tool first needs the path resolved and the file's data located. The metadata path supplies names, attributes, and file-to-object mappings; the data path retrieves payloads from caches or object storage. Metadata services can batch requests and cache records [16, 17]; immutable distributions can also organize records for demand fetching [2, 14]. Thus a file-system operation need not generate a remote metadata request.

For a large fixed collection of small files, the metadata database may still serve repeated reads alongside mutable updates. Moving that collection's read service to immutable object pages can reduce this shared load, but requires first fixing and publishing a version. Its value therefore depends on reuse and update frequency as well as query latency. The evaluation measures database pressure directly; a high FUSE metadata request count alone does not establish such a bottleneck.

The requested file range also differs from the storage work it induces. Kernel caching and readahead affect which requests reach FUSE. The client then maps those requests to stored objects and may fetch a range or an entire object. Backend encoding and integrity checks can require access to still different units [1, 5]. Writes similarly pass through buffering and object formation before publication. Thus a small file request does not by itself determine transferred bytes, verification work, or uploaded object size. The following observations characterize requests received by FUSE handlers.

2.3 Observed requests in real agent runs

Capture and scope. We reanalyzed eight SWE-bench Verified task runs [6]: five Core and three Exploratory. All used the same agent harness configured for `deepseek-v4-flash`. Exploratory runs used a broader investigation/testing prompt and a 30-minute budget; all finished earlier. Each base-commit checkout was mounted through an instrumented passthrough file system on Ubuntu 24.04 under WSL, recording actual FUSE handler requests. The companion record retains the harness and instrumentation versions. Streaming reanalysis covered all 1,138,884 events: 217,129 Core and 921,755 Exploratory. Counts retain every status, 64 setup/canary requests before agent startup, and teardown, including 24 requests after recorded run finish. Group statistics weight requests equally, not tasks. Per-task counts and source-validation details are in the companion workload record.

Metadata traffic and read dominance. The metadata methods—`lookup`, `getattr`, `getxattr`, `readdirplus`, `opendir`, and `releasedir`—accounted for 61.5% of Core and 56.4% of Exploratory calls (Figure 1a). Other comprises all methods except these six and read/write. Restricting the denominator and numerator to `ok` events reduces these shares to 52.0% and 44.2%: failed probes materially change the request mix. Across tasks, the all-status share ranged from 39.6% in SymPy to 65.4% in Django, so the group majority does not describe every task. Read calls outnumbered writes by 19.9:1 in Core and 4.16:1 in Exploratory; per-task ratios ranged from 3.12:1 (Pytest) to 42.7:1 (Matplotlib). These observations motivate evaluating both positive and negative metadata paths alongside data I/O.

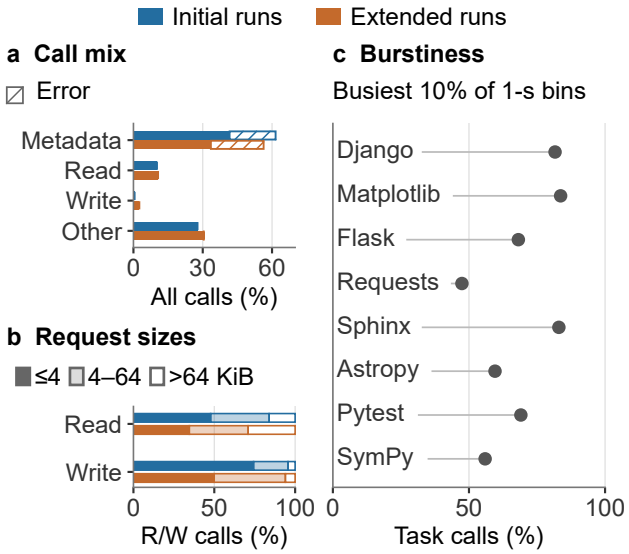


Figure 1: FUSE request structure across eight task runs. Initial and Extended denote the five Core and three broader-prompt Exploratory captures. (a) All-call shares; hatching marks errors. (b) Requested-length distributions within method/group. (c) Each task's call share in its busiest 10% of one-second windows.

A mixture of request sizes. All 22,142/98,292 reads and 1,112/23,600 writes (Core/Exploratory) were `ok`; 1 KiB = 1024 bytes. Reads ranged from 4 to 128 KiB, with Core and Exploratory medians of 8 and 16 KiB. Writes ranged from one byte to 128 KiB, with medians of 4,084 and 4,117 bytes, respectively. Requests of at most 4 KiB accounted for 47.7% and 34.5% of reads, and 74.3% and 49.9% of writes. The interval $(4, 64]$ KiB contained a further 36.2% and 36.4% of reads and 21.3% and 44.0% of writes (Figure 1b). This mixture motivates measuring how client fetching and uploads transform requests before testing backend layouts. FUSE observations occur below kernel caching and readahead; they do not identify application syscall sizes, unique bytes, finalized object lengths, or an optimal block size.

Task and temporal heterogeneity. Tasks generated 8,542–634,726 calls. Astropy contributed 68.9% of Exploratory calls, versus 6.0% for Pytest. Capture spans, from first to last request start, were 42.7–71.3 seconds for Core and 119.9–556.1 seconds for Exploratory. Windows include zero-call and final partial windows. The busiest 10% of one-second windows, rounded up per task, contained 47.4–83.7% of its calls (Figure 1c). The companion record retains the full method breakdowns, size ECDFs, and per-task cumulative time curves. Replay should retain these per-task mixes and bursts. Each task was observed once; different repositories, prompts, and budgets prevent a causal interpretation of the group contrast. Without tool-phase labels, bursts cannot be assigned to searching, editing, or testing.

Status semantics and evidence limits. The capture contained 872,519 `ok`, 253,669 `error`, and 12,696 `incomplete` events. Errors occurred in `lookup` (233,452), `getxattr` (20,203), and `rmdir` (14). Collection notes associate many with failed path probes or unsupported attributes; absent errno values prevent event-level cause attribution. The parser labels a request `incomplete` when no recognized completion is paired with it: here, 12,688 `forget` and eight `destroy` records. Neither label counts failed agent tasks. All eight agent and daemon exit codes were zero, which does not establish SWE-bench grading success.

Handler latency measures WSL passthrough intervals and excludes model response time. Elapsed capture time instead includes inter-request gaps, which may contain model waiting; summing overlapping handler intervals cannot recover task wall time. These observations do not measure RustFS, Redis, network object storage, or cloud-disk latency. They motivate metadata and mixed-I/O experiments for this single-agent/model panel, but do not establish cross-task sharing, sparse mutation, or performance gains from BrewFS, RustFS, or the automatic hint policy. Those remain questions for Section 8.

2.4 Implications for storage design

The observed mix motivates testing positive and negative metadata queries, mixed data-request lengths, and task-specific bursts. It does not select a metadata layout or an object size. Grouping can amortize requests when nearby records or bytes are reused, but scattered accesses can fetch unnecessary data. Measuring locality and version reuse must therefore precede any claim that access savings repay construction and retention. Section 3 defines the state and access interfaces needed to compare these choices while preserving each task's file view.

3 Overview and System Contract

3.1 From file operations to storage choices

To read `src/parser.py`, a client resolves its name, obtains the visible inode and data mappings, and fetches the requested bytes. An overwrite stores new payload and updates the file's metadata under the selected durability contract. BrewFS separates three decisions along this path: which state the task sees, how immutable metadata is retrieved, and how payloads are divided into storage units. On the object path, g denotes client block size, B denotes backend erasure-coding block size, and q denotes the physical verification/read quantum.

Composition rests on two interfaces. The lower reader resolves names, attributes, and data locators under a fixed

version identity; the data path returns the referenced logical bytes under the object's persisted layout. Changing the lower representation must preserve the first contract, and choosing a storage layout must preserve the second.

3.2 Pinning state and isolating changes

Let S identify a shared immutable baseline and Δ_i the private writable state of workspace i. Its view is

$$W_i = \text{Resolve}(S, \Delta_i). \tag{1}$$

Workspace creation pins S so that subsequent lower queries refer to the same version. The baseline exposes name, attribute, and data-location queries through either a sealed native layer or a packed manifest. It can cover a selected subtree without freezing an entire source volume. Following a mutable latest pointer on each lookup would not preserve this fixed starting state.

The isolation invariant is view-local mutation: an operation in W_i leaves S and another workspace's private state unchanged. Private names, attributes, deletion markers, and data mappings determine which inherited records remain visible. Section 4 specifies resolution and the operation boundaries that preserve inode identity and prevent stale bytes from reappearing. Process isolation and merging independent edits require separate runtime or application protocols.

3.3 Reading, writing, and publishing

Figure 2 follows these parallel interfaces. An inherited read obtains a locator from the chosen lower reader, then fetches its payload range. An overwrite publishes a private mapping to new upper bytes; other workspaces retain their references to S.

Writes introduce two visibility boundaries. A dirty buffer can serve this client's reads before remote durability. Publishing a recoverable version requires its referenced objects to satisfy the selected persistence contract and its metadata to be validated.

Workspace and reader ownership must retain referenced metadata and payloads until reclamation is safe. A published version may contain application checkpoints; restoring process memory and open handles remains the runtime's responsibility. Section 7 identifies the current mount and recovery paths.

4 Delta-Based Workspaces

4.1 Creating a view and resolving names

A workspace binds an immutable lower S and private writable upper Δ_i, yielding $W_i = \text{Resolve}(S, \Delta_i)$. Creation registers the base identity and private control state without

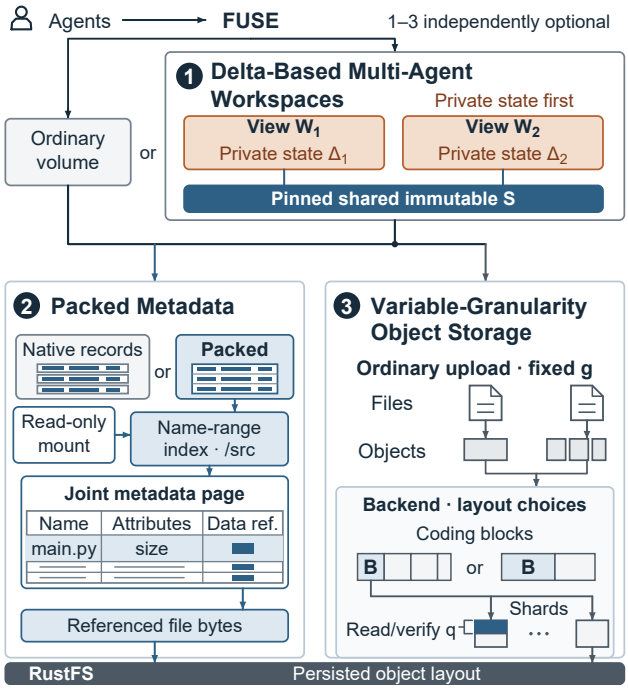


Figure 2: Independently optional mechanisms. Packed pages co-locate multiple metadata records. Ordinary uploads use fixed g; the backend diagrams show alternative B layouts for eligible objects. q applies within a shard. Layouts are schematic; producer contracts and admission gates differ (Section 7).

copying the complete metadata tree. The catalog path identifies the base by layer, sealed version, and root hash; a packed lower additionally binds its authenticated manifest. Binding validation, ownership records, and mount preparation still contribute to creation cost.

A name lookup first consults the private record, which distinguishes a present entry, an explicit deletion, and absence. For parent directory d and name n, let $H_i(d,n)$ indicate that an applicable rule blocks inheritance of that lower name. Then

$$R_i(d,n) = \begin{cases} e, & \Delta_i(d,n) = \text{Present}(e), \\ \bot, & \Delta_i(d,n) = \text{Whiteout}, \\ \bot, & \Delta_i(d,n) = \text{Absent} \wedge H_i(d,n), \\ R_S(d,n), & \Delta_i(d,n) = \text{Absent} \wedge \neg H_i(d,n). \end{cases} \tag{2}$$

Here $\bot$ denotes absence. H_i covers directory-opacity or other lower-inheritance rules without hiding a present upper entry. A whiteout terminates lookup; deleting an inherited name must preserve this marker, since merely removing a private record could expose the lower name again.

Directory enumeration applies the same precedence to each name, removes duplicates, and returns attributes of the

visible inode. The catalog-backed buffered path and a direct-I/O workspace path both retain an inherited file's inode number when materializing private attributes or mappings. Updates through one hard-link alias therefore remain visible through the others.

The direct-I/O path uses a transactional private namespace. Rename locks participating inodes in number order, validates the observed rows and writer authority, then atomically commits the source whiteout, destination binding, and affected inode and parent records. Its namespace reads validate an operation-local view and retry conflicts: a successful read can observe either side of a concurrent rename, but not its partially updated rows. Separate calls may straddle that commit. The catalog-backed buffered path instead serializes local mutations and uses version-guarded catalog transactions. These mechanisms do not establish a snapshot across multiple system calls; entry-specific caching and runtime bounds remain as described in Section 7.

4.2 Resolving overwrites and truncation

An overwrite changes which bytes a file range references. The buffered path records private data extents alongside namespace and inode changes. The direct-I/O path represents each changed logical chunk by a normalized map with explicit spans of upper data, lower references, or zeros. These respectively identify newly stored bytes, inherited bytes in S, and holes. An explicit map covers its applicable range completely; an uncovered interval is not an implicit lower reference.

A short overwrite replaces its mapped interval while preserving lower references elsewhere. In this path, truncation first commits the reduced size, a nonincreasing lower-visible boundary, and a cleanup intent. Cleanup replaces discarded spans, including existing upper data, with zero spans; subsequent extension completes pending cleanup before changing the size. For example, writing upper bytes at $[6,10)$, shrinking to 7, then extending to 34 must yield zeros at $[7,34)$. Lower-visible size alone would not invalidate the old upper bytes. These steps describe the direct-I/O path; other mounts retain their own update protocols. Object formation remains separate: a short logical write need not produce a short object.

The buffered client's dirty overlay makes its bytes readable before writeback completes. If local dirty data covers $[4,8)$ KiB, a read of $[2,10)$ KiB combines two committed 2 KiB ranges with the dirty 4 KiB range. A buffered workspace can use both this overlay and persistent private mappings; local read-your-writes alone establishes neither remote durability nor a published recovery point.

4.3 Publishing and retaining a version

The direct-I/O workspace uploads an object durably before publishing its mapping. The ordinary buffered path defaults to upload-before-commit and offers explicitly selected commit-before-upload mode, which can expose mappings before their objects are remotely available. Preparing a recoverable version therefore requires draining and validating referenced objects even when local reads already return new data.

A worker handoff requires application checkpoints, stopping new source writes, and making required payloads and metadata durable before publishing the immutable version. The destination binds that version and acquires write authority; lease and generation checks reject stale writers. Applications restore execution state and reopen files from this recovery point. Section 7 distinguishes implemented lifecycle components from connected recovery paths.

Publication must preserve prior versions still held by workspaces or readers. Reclamation separates those references from abandoned construction objects. For one shared baseline, live logical metadata obeys

$$M_{\text{live-meta}} = M_S + \sum_i M_{\Delta_i} + M_{\text{control}}. \tag{3}$$

M_S counts the shared metadata once; M_{Δ_i} counts private names, attributes, deletion records, and mappings; M_{control} counts disjoint binding and ownership records. Upper payload, encoding and checksum overhead, unreclaimed objects, and caches belong to separate physical accounting. Complete private records and mappings grow with modification and deletion; comparison with materialized workspaces must match visibility and durability contracts.

5 Packed Metadata

5.1 Organizing immutable records for reuse

Large collections of small files keep names, attributes, and data locators in the metadata database even when their contents no longer change. Concurrent consumers query those records alongside mutable workspace operations. Packed Metadata moves the fixed version's read service to indexed object pages and client caches; private updates and ownership still use metadata and control services. The intended benefit is lower central database traffic and interference, purchased with construction, publication, and object-access work.

Packed metadata places directory entries, common attributes, and file-data locators in bounded object-resident pages. The resulting snapshot describes a selected subtree and serves either a standalone read-only mount or a workspace lower, with demand-driven access.

A logical read group covers one parent directory and a range of raw entry names in byte order. Its joint metadata

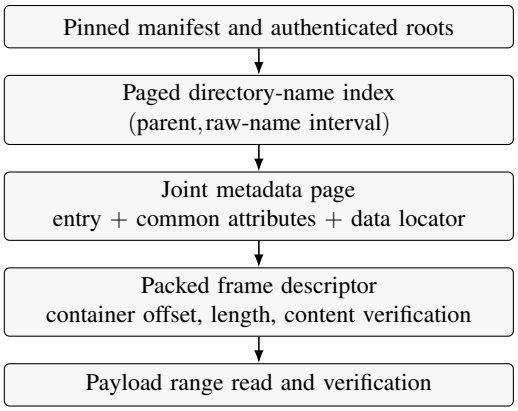


Figure 3: Representative packed-snapshot lookup/read path. A name lookup supplies joint metadata and a data locator; reading referenced data resolves authenticated descriptors before fetching and verifying payload ranges.

page co-locates each entry with common attributes and data locators, so a name lookup can supply information needed by subsequent attribute and data operations. Neighboring names may reuse the same page. The raw page is bounded at 256 KiB, while a physical container may hold several groups. This separates the unit of metadata organization from the object that stores it. Range indexes route requests to these groups; their height depends on page capacity and record count, rather than on the source directory tree's depth.

5.2 Querying a pinned snapshot

A reader pins a manifest identity and starts from its authenticated index roots. To look up a name within a parent directory, the directory-name index identifies the group covering that key. The reader resolves its container reference and fetches the joint metadata page's byte range. It verifies the stored-byte digest before decoding, then checks the decoded entry count and name boundaries against the authenticated group reference. The matching record supplies the inode, common attributes, and data locator (Figure 3).

The query input determines which index is needed. Attribute queries addressed by inode use a separate index, while enumeration traverses ordered name ranges and uses counted indexes for directory-ordinal navigation. A file read returns inline contents or follows a data locator to an authenticated descriptor, fetches the referenced payload range, and verifies it. Packed data frames define snapshot-level access and integrity units; the backend serves these object requests using its own persisted layout.

Supported snapshot queries obtain names, attributes, and data locations without per-file records in the mutable metadata database. This lower-query independence leaves private upper checks, mutation, and lifetime coordination to the workspace and control protocols.

5.3 Preserving identity and authenticated results

Every returned record or payload must connect through authenticated references to the pinned manifest. Digest and key-boundary checks prevent accepting the wrong records for a query. A missing referenced object or failed authentication is an error, not evidence that a name is absent.

Authenticated records remain valid under their pinned identity after later versions are published. Cache reuse still depends on locality: a cold lookup may traverse several index pages, and data reads may need descriptor and payload requests beyond the metadata page. Co-location therefore offers reuse, not a fixed number of GETs per operation.

5.4 Publication, lifetime, and amortization

Construction needs a consistent source view, established through a snapshot or validation protocol; output immutability alone cannot establish it. The producer groups records, uploads referenced objects, and produces the indexed manifest. Publishing its binding separately requires those objects to be durable and valid. Reclamation preserves objects needed by retained versions or readers. Rebuilding creates another version; ordinary file close does not rebuild a snapshot.

Native overlay fork does not require packing. Its first child pays for the native consistency and version-binding steps; a packed child must also wait for any required construction and publication. Later children can reuse that published representation. These are distinct costs. Offloading queries can help while source records remain retained, but database space falls only if the lifecycle protocol can safely retire them. Replacing reads with object requests does not itself reclaim records.

The access benefit depends on how much of each fetched page is useful. Repeated nearby lookups can reuse joint metadata, whereas isolated lookups may transfer records they never consume. Inlining file contents, compressing pages, and co-packing payload regions also affect transfers, decoding, and cache use. Their costs must be separated from the effect of metadata co-location.

For one version, K counts uses: one assigned workspace lifecycle or one fixed standalone query session, including failures and timeouts. These units are reported separately. The accounting interval T spans construction through reclamation, including retention after the last use. Each additive resource metric c is separate:

$$C_c(K,T) = C_{c,\text{build}} + C_{c,\text{publish}} + \sum_{i=1}^{K} C_{c,i} + C_{c,\text{retain}}(T) + C_{c,\text{reclaim}}. \tag{4}$$

Build and publication include validation; each use includes setup and access. Retention includes background main-

tenance and, for monetary cost, storage charges such as $\int_0^T r(t)V(t)\,dt$, where $V(t)$ is billable resident bytes and $r(t)$ is currency per byte per unit time. Thus equal K does not imply equal cost. Terms use one unit—CPU-seconds, transferred bytes, or currency—throughout; concurrent task durations are not a batch makespan. Shared work is charged once across uses and versions; a shared object's retention uses the union of its residence intervals. Reuse pays only when setup and access savings exceed the additional construction, publication, retention, and reclamation costs relative to the matched baseline; churn and weak locality can erase that benefit.

Immutable object-backed metadata already has strong precedents [2, 14, 23]. We will compare the joint layout with immutable sorted-KV records at equal content and cache budgets, and with compatible demand-loaded immutable-image systems. Standalone read-only and workspace-lower use are separate comparisons. The sorted-KV control tests layout organization and is not a reproduction of DeltaFS.

6 Variable-Granularity Object Storage

A file read names a logical byte range, but its cost depends on the objects containing that range, the bytes fetched, and the storage units verified. Large objects can reduce request count for sequential access while increasing the work needed to serve a partial read. BrewFS therefore separates client object formation from backend encoding and verification. The connection between them is the finalized object, whose length becomes known after assembly, applicable compression, and framing.

6.1 From file ranges to objects

The ordinary data path maps file offsets to logical chunks and resolves the visible slices within each chunk. A chunk is an addressing range; slices identify the written content that supplies its visible intervals. The uploader partitions each slice's data using the configured client block size g and stores the resulting blocks under the slice identity and block index. A 10 MiB slice with $g = 4\,\text{MiB}$, for example, can produce raw blocks of 4, 4, and 2 MiB.

Compression, headers, and framing then determine each object's finalized upload length L_o. Thus g controls the partition of the slice, whereas L_o describes the bytes actually uploaded. Flush boundaries and partial writes affect which slices and objects form, and a short tail need not occupy a full block. When reading, the client uses the same geometry to translate a visible slice interval into object references and offsets. A new slice can replace one interval while untouched intervals continue to reference older objects; the replacement need not rewrite the complete old object.

Chunk size is a nonzero multiple of g. Readers reconstruct block addresses from this configured geometry, so changing g for existing data requires a compatible representation or migration. The design exposes configuration choices, not an access-driven sizing controller.

6.2 Mapping objects to backend storage

Once L_o is known, a compatible RustFS research backend can use it to inform a separate layout decision. Let B denote the backend's erasure-coding block size. The client may request B through an encoding-block-size hint $h(L_o)$, without changing g or multipart part sizes. The backend validates the request under its compatibility and eligibility rules and persists the effective B. Omitting a hint retains default selection. Multipart initiation uses the complete object's length, and retries retain that object's decision.

The interface supports fixed hints of 64 KiB, 256 KiB, 1 MiB, and 4 MiB, or a length-based hint policy. These act on the formed object; they do not select the client block size. Section 7 describes when the current policy leaves the backend default unchanged.

The physical verification/read quantum q determines how much shard data must be verified together. For d data shards and p parity shards, let s be one complete stripe's logical shard size. A separately enabled backend policy can select a physical verification/read quantum for eligible new framing subject to

$$q \mid s, \qquad qd \mid B. \tag{5}$$

These are admission constraints for the new framing, preserving stripe alignment while permitting smaller verification units. Both hint admission and new q framing have their own compatibility gates. Reads and healing use persisted object geometry even after write settings change.

6.3 From an object range to physical work

Object size alone does not determine the bytes fetched for a file read. Range GET can retrieve part of an object when its format permits independent access. The ordinary store supports page-aligned range reads for eligible small requests; compressed blocks or a whole-object fetch policy can require larger transfers. A packed frame provides a decoding and integrity boundary inside a snapshot object, distinct from both the client block size g and the backend quantum q. Caching and concurrent-read coalescing determine which of these requests reach the backend.

The backend maps the resulting object range into shard intervals. For a shard interval $[a, a+\ell)$, with $a \geq 0$, $\ell > 0$, and $q > 0$, the number of verification/read quanta intersected is

$$n_q(a,\ell) = \left\lceil \frac{a+\ell}{q} \right\rceil - \left\lfloor \frac{a}{q} \right\rfloor. \tag{6}$$

Ignoring tails and batching, the covered data is approximately $n_q q$ plus checksums, summed over shards actually

g	Object count	Whole-object bytes fetched
1 MiB	64	2 MiB
4 MiB	16	4 MiB
8 MiB	8	8 MiB
16 MiB	4	16 MiB

Table 1: Idealized object formation and a 2 MiB partial read. The last column assumes complete fetches of the intersecting objects, with no cache hits or compression and ignoring framing overhead; it does not describe every range-read path.

read. The application request cannot be substituted directly for ℓ: the client range, compression, and coding determine that interval. Smaller q can reduce over-read while increasing checksum overhead. For fixed (d, p), the nominal redundancy remains $(d+p)/d$; a large B does not imply a fully padded tail stripe.

6.4 Costs and scope

The tradeoff in g is visible even before backend encoding. Table 1 considers one uncompressed 64 MiB slice starting at file offset zero and a read of $[10, 12)$ MiB. Increasing g reduces the number of objects in the slice, but increases bytes fetched for this read if complete objects must be obtained. These are arithmetic examples, not measurements or a policy recommendation.

These layers require separate cost accounting. Network read amplification compares object response bytes with application bytes served; upload amplification compares uploaded bytes with application bytes written; physical read amplification compares storage-device bytes with application bytes served. Request counts, compression work, and cache occupancy capture additional costs. Compaction, retries, and accumulated short slices can increase upload work, so smaller objects do not guarantee lower write amplification. Similarly, whole-object fetching can prevent a smaller q from reducing the work generated by a partial application read.

Snapshot producers and workspace upper writers retain their own formats and upload contracts. Section 7 identifies their coverage and the ordinary client's coupling of enabled hints with fetch and cache policy. Those client effects must be separated from persisted geometry when attributing a benefit to B or q.

7 Implementation Boundaries

The implementation connects the mechanisms through several access paths. Their differences affect both guarantees and performance attribution. The following boundaries come from source inspection; the companion record identifies the inspected revisions, while runtime and failure validation remain part of the evaluation plan. These are boundaries of the audited revisions; the execution build must be qualified again before using them to admit or exclude an experimental configuration.

7.1 Buffered and direct access

An ordinary volume resolves mutable metadata and sends file data through buffered caching and writeback. A workspace can reuse this path while replacing metadata lookup with private-state resolution over a sealed catalog or packed lower. A standalone snapshot reader instead provides read-only access without constructing a private upper. Its metadata cache is separate from payload caching; the current read-only path does not admit configurable payload-cache or prefetch budgets.

A second workspace implementation stores current private namespace and mapping records transactionally and uses a dedicated direct-I/O adapter. This is the concrete namespace and truncation path described in Section 4. It bypasses ordinary buffered writeback and metadata TTL caching. Its operation sizes, complete-directory enumeration, and live ownership set have admission bounds. It currently supports newly created workspaces, without process-crash takeover. These limits belong to that path, rather than the logical workspace model or every mount.

The direct path authenticates an entire upper object before returning a subrange. Thus a smaller backend verification unit need not reduce work for a short application read. Comparing this path with a buffered volume changes cache and fetch behavior as well as workspace representation; an ablation must retain one access path or control those changes explicitly.

7.2 Publication and recovery

Snapshot publication validates uploaded objects before installing the lower binding and acquiring reader ownership. Recovering a binding also requires generation and lease checks: knowing a manifest's identity does not grant authority to resume writes. One connected recovery path reuses the same persistent storage allocation and checks its worker identity and scoped metadata authority. It does not provide arbitrary-worker migration.

Catalog lifecycle operations encode quiescence, object verification, retention, and atomic version transitions. Connecting these operations to worker handoff still requires runtime orchestration. The recovery target is a published file state; applications must checkpoint execution state and reopen tools separately. Source-level lifecycle operations therefore do not by themselves demonstrate end-to-end recovery.

7.3 Object formation and layout admission

The ordinary uploader partitions slices using fixed volume geometry, defaulting to $g = 4\,\text{MiB}$ within 64 MiB chunks. Readers use the same geometry; an incompatible configuration is rejected. The proposed 1/4/8/16 MiB settings expose a design space, not an adaptive sizing policy.

Backend hints are optional and require both client support and compatible storage servers. Hint admission and new verification framing have separate enable and fleet-compatibility gates, disabled by default. The backend checks eligible writes and persists their effective layouts, which govern later reads and repair. Changing admission settings does not reencode existing objects.

Uploader coverage matters. The ordinary uploader propagates hints; direct workspace writes and the current snapshot producer use default settings. Extending them must preserve conditional object creation and rejection of conflicting writes. Direct workspace uploads also form objects from write fragments, rather than ordinary buffered blocks, and cap those fragments at the automatic policy's no-hint threshold.

That policy omits hints for $L_o \leq 1\,\text{MiB}$ and requests $B = 1\,\text{MiB}$ otherwise; the inspected backend already defaults to $B = 1\,\text{MiB}$. Enabling hints can therefore leave the stored layout unchanged while changing client fetch and cache policy. Experiments must record actual (g, L_o, B, q) and physical work, and separate those client effects from backend geometry. These integration restrictions constrain current comparisons without making any of the three mechanisms mandatory.

8 Evaluation Plan

The evaluation first isolates the three optional mechanisms, then tests what they contribute to branching, migration, and concurrent tasks. Five questions organize the planned experiments. No system-performance results are reported here. The eight-run observations in Section 2.3 and the seven later replay archives are separate datasets.

8.1 Systems, inputs, and accounting

The current external comparator is JuiceFS [7], using native cloning and shared read-only directories where appropriate. Both systems use the same four-server object backend at separate times, with isolated namespaces and matched client, metadata, cache, and persistence budgets. The current upstream file-system revision is frozen at execution start; mechanisms and restart semantics are qualified on that build. Research-backend extensions and internal ablations are identified separately. This campaign does not establish superiority over all immutable-distribution systems.

The companion protocol freezes byte generators, query lists, branch trees, arrival schedules, targets, deadlines, and repetitions. Inputs include a 1 GiB tree with 32,896 files; a 1 GiB indexed record collection; and a small-file tree with 32,768 or 1,048,576 one-KiB files, with a larger resource-bounded extension. Seven complete file-operation traces cover code tasks, build/test, and archive/indexing work. Two are captures of the same Pytest task; replay copies do not increase task diversity.

We measure preparation, execution, validation, and reclamation separately, and include them in lifecycle accounting. Shared objects are counted once. Resource metrics in $C_c(K, T)$ retain their units; concurrent task durations are not summed into batch elapsed time. The existing container environment supports direct process control without a Kubernetes controller. Actual Agent-session validation is separate from deterministic file replay.

8.2 Overlay fork and retained branches

RQ1. What does overlay contribute without packing metadata? A shared operation suite first compares the ordinary volume, overlay with native metadata, and overlay with packed metadata, holding object layout and payload caching fixed. This measures lookup, read, write, and sync overheads even when no branch is created. A volume without a fork API is not assigned a zero fork cost: a materialization adapter must provide the same isolated child state and include its preparation work.

A parent first changes data and namespace entries and confirms a version. We then fork 1, 16, or 64 children, each with private modifications and a saved state. Children must inherit the parent's changes while leaving the parent and siblings unchanged. Clean-baseline creation is a separate case. A fixed 32-fork budget also compares a star, a four-way tree, and four chains of depth eight, retaining 33 states in each case.

The internal control materializes current metadata under the same API, payload-sharing, cache, and durability contract. JuiceFS may share data blocks through its native clone; it is not forced to copy all payloads. We report parent preparation, request-to-first-correct-operation time, first durable edit, save latency, private metadata, and uploaded bytes. Retention tests create 1,000 and 10,000 recoverable states, with 100,000 as an extension under fixed time and memory limits. Reopen checks and reclamation after deleting half distinguish retained state from discarded records. Admission failures and outstanding cleanup remain results.

8.3 Packed metadata and database load

RQ2. When does offloading fixed-version metadata repay packing? The central comparison is native overlay against the same overlay using a packed lower of identical content. Both use a consistent fixed source; this avoids attributing immutable caching benefits solely to packing. Standalone read-only comparisons isolate representation without requiring a

workspace.

We separate three paths: the first native fork, including native version capture and binding; the first packed fork, additionally including required construction, upload, validation, and publication; and subsequent forks from each already published version. The first useful child and the full batch are timed directly. Pipelined phases may overlap; their durations are not added to manufacture a critical path. Reuse does not erase the initial construction bill.

On the small-file trees, each query session performs 32,768 operations. Workloads separate random positive lookups, mixed positive/negative lookups, inode attributes, neighboring-name access, directory enumeration followed by attributes, and lookup followed by a file read. One, 16, or 64 consumers access overlapping or disjoint paths. Cold, index-warm, and metadata-warm cases have equal declared cache budgets. Inlining and compression are disabled for the representation control and enabled only in separately reported practical configurations.

Measurements distinguish lower-database, private-upper, and control requests from object-index, joint-page, descriptor, and payload requests. We report database CPU, memory and persistent bytes, query latency, transferred bytes, decoding and verification work, and object-service and client resources. A fixed-rate mutable-workspace probe shares the metadata service while identical background read schedules target native or packed versions. Its queue, deadline completion, and tail latency test whether removing fixed-version queries also protects mutable operations. Database relief is evaluated alongside work transferred to other services.

Query offload does not imply database-space reclamation. We distinguish retained source records from versions whose source can be safely retired; unsupported retirement is reported. An additional immutable per-record control can separate database offload from joint-page organization, but only after an equivalent implementation is available.

Each version serves K query sessions, with reuse counts of 1, 4, 16, 64, and 256. We then publish a new version with 0.1%, 1%, or 10% changed files. Eight version cycles expose rebuilding cost; no incremental packer is assumed. Post-use retention is 0 or 600 seconds and is included in T. For each resource metric we compare $C_{c,\text{packed}}(K,T)$ with $C_{c,\text{native}}(K,T)$ and report observed break-even regions. Few consumers, rapid updates, weak locality, and warm native caches can favor native overlay; a break-even point outside the measured range is not claimed.

8.4 Object formation and backend work

RQ3. Which granularity choices reduce storage work, and at what write or recovery cost? First, identical logical writes use supported client sizes $g = 1,4,8,16\,\text{MiB}$ at fixed backend geometry. Second, fixed objects and S3 request streams compare admitted backend B hints and q policies. Finally, range and whole-object fetching are compared on the same objects with cache, coalescing, and prefetch controlled. Persisted layouts are inspected; an automatic policy that leaves the layout unchanged is a no-treatment control, not a benefit from adaptation.

Indexed 4 KiB reads with shared or disjoint keys, boundary-crossing reads, full scans, overwrites, and supported degraded reads expose different trade-offs. Object counts, finalized lengths, HTTP bytes, device-read bytes, checksum/encoding CPU, remote-write completion, and repair traffic connect the mechanism to tool time. A configurable client size does not establish an access-driven sizing controller. Missing physical counters remain missing rather than being inferred from HTTP traffic.

8.5 Migration and recovery

RQ4. Do the representations reduce migration cost, or merely move it into earlier publication? Sixteen workspaces run on four source workers; four target workers are already available. We migrate 0, 1, 4, or 8 selected workspaces while the others continue. A larger configuration migrates subsets of 256 workspaces. Native and packed paths are measured separately, with paired no-migration runs.

At a completed-tool boundary we test both an already confirmed version and pending modifications. The latter includes draining, capture, and any required packing and publication after the migration request. Queueing counts toward interruption. JuiceFS reopens the existing remote directory; it is not forced to clone or copy the tree. Targets are tested cold and warm. The coordinator records request-to-first-correct-operation time, write-authority transfer, target access, full continuation cost, transferred bytes, and unaffected neighbors' deadline completion and tail latency. Container handoff is called cross-node migration only when node identities establish that distinction.

Separate failure tests interrupt before and after confirmation, disconnect the source from coordination while preserving storage access, and fail the target during recovery. Independent readback verifies committed content and attempted stale-writer changes. Process termination, application publication checks, and storage-enforced rejection are recorded separately. A synthetic state file verifies file-state handoff, not Agent-session recovery. Actual session tests fork or move a saved Agent after a complete tool result, preserving history and remaining budget, and require a valid subsequent tool result and final task grading.

8.6 Concurrent tasks and analysis

RQ5. Do mechanism savings improve useful capacity at fixed resources? The fixed-work replay panel contains eight copies of each of seven traces: 56 assignments at concurrency 1, 4, or 16 on four workers. A common observation

deadline includes queueing. The scale study instead uses 4, 16, 64, or 128 workers, four execution slots per worker and seven assigned traces per worker, at fixed backend capacity. Its growing workload supports throughput and per-case latency comparisons, not a fixed-work speedup. Actual node placement and replay-engine CPU limits are recorded.

Internal comparisons change one supported mechanism within a matched path; unsupported combinations are not invented. Composition is compared with its best supported proper subset, and external system advantage is a separate comparison. Zero interaction does not refute useful component gains. Three initial independent paired batches per cell yield raw paired outcomes and ranges; stronger inference requires a sample size frozen from a separate variance pilot. Requests and sibling tasks are not independent replicates. Failures remain in assigned-work denominators, complete-success makespan is reported only when all assignments succeed, and deadlines, failed outcomes, and measurement gaps remain explicit. Replay and real Agent completion are reported separately.

9 Related Work and Scope

Workspaces and agent-native file systems. OverlayFS provides layered namespaces with a private upper and shared lower [12]. BranchFS applies copy-on-write branch contexts to agentic exploration, with a commit/abort lifecycle described through a workshop prototype and a proposed kernel abstraction [18]. YoloFS addresses visibility and control over agent effects through staging, snapshots, and permissions [24]; AgentFS documents a file-system and tool-state abstraction for agents [3]. These systems establish the workspace mechanisms on which this work builds. BrewFS focuses on remote metadata and object access; process isolation and approval of agent actions require separate runtime policies.

The comparison must distinguish a local private view from a remotely durable one. OverlayFS and BranchFS are useful controls for workspace creation and mutation; their persistence boundaries must be matched or reported separately. JuiceFS supplies an object-backed volume baseline with a metadata service and chunk/slice/block data organization [7]. These comparisons test the costs of inherited versus materialized metadata, private updates, and object access, rather than treating the presence of a private upper as a new mechanism.

Namespace and metadata organization. TableFS, IndexFS, LocoFS, and InfiniFS organize, partition, and cache metadata to change the work required by file-system operations [11, 13, 16, 17]. Recent systems also study hierarchical object metadata and distributed update coordination [10, 21]. Their access paths make record layout, batching, and cache reuse relevant comparison criteria: a metadata service need not issue one remote request per file operation.

DeltaFS is a particularly close conceptual predecessor. It uses private namespaces and immutable mutation logs over shared storage, with selective composition of prior state [23]. BrewFS builds on the separation of shared state and private changes. Its current-value upper and Packed Metadata's directory-oriented groups are concrete representation choices to compare with immutable logs and sorted records. An internal sorted-KV control can isolate co-location and indexing costs; it cannot establish superiority over the DeltaFS system or replace a compatible system-level comparison.

Immutable distribution and storage backends. CernVM-FS provides demand-driven software distribution with indexed catalogs, while Nydus describes metadata/data organization for lazy image access [2, 14]. Slacker and DADI also address image distribution and startup costs [4, 9]. A strong composition alternative combines a mature immutable lower with a compatible writable upper and object backend. It already shares state and fetches it on demand. A direct comparison would need to measure useful records per fetch, excess payload transfer, and construction costs at equal content, cache capacity, and persistence. The current campaign uses JuiceFS as its external comparator and does not establish an advantage over these immutable distribution systems. Standalone lower access and complete writable workspaces require separate comparisons; neither the proposed interfaces nor their composition establishes an advantage by itself.

Distributed systems such as Ceph and Tectonic separate aspects of metadata and data services [15, 20]. Erasure-coded storage research considers repair cost, changing redundancy over a file's lifetime, and the relation between data placement and redundancy [1, 5, 8]. Okapi's separation of striping and redundancy grouping is different from our (B, q) distinction, although both motivate making coupled decisions explicit. Client block granularity g additionally determines how file data forms objects; it is distinct from backend coding and physical verification granularity. Our comparisons hold the code rate fixed and separate object counts, checksums, tails, and repair work. A smaller unit can reduce over-read without reducing the nominal redundancy ratio, and can increase checksum or repair cost.

Scope and unresolved evidence. Poor name locality, high mutation, rapid version churn, and large sequential reads can erase the expected savings. The current entry points and lifecycle support further bound the comparisons (Section 7). The planned evaluation must establish where each mechanism repays its cost and whether a supported combination improves on its best supported subset and the external comparator.

10 Conclusion

BrewFS separates a shared version's identity from the metadata and object units used to serve it. Its workspace, packed-metadata, and object-layout mechanisms expose different opportunities to reduce storage work, subject to visibility, publication, and lifetime constraints. The eight-task FUSE study characterizes request structure, and the source audit identifies available runtime paths. The remaining evaluation must test whether metadata-database relief and access savings repay lifecycle costs under matched contracts, and how these affect branching, migration, and concurrent tasks. The planned external comparison uses JuiceFS. The present evidence characterizes the workload and implementation scope; it does not establish a system-performance gain.

References

[1] Sanjith Athlur, Timothy Kim, Saurabh Kadekodi, Francisco Maturana, Xavier Ramos, Arif Merchant, K. V. Rashmi, and Gregory R. Ganger. Okapi: Decoupling data striping and redundancy grouping in cluster file systems. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 897–914. USENIX Association, 2025.

[2] Jakob Blomer, Carlos Aguado-Sánchez, Predrag Buncic, and Artem Harutyunyan. Distributing LHC application software and conditions databases using the CernVM file system. *Journal of Physics: Conference Series*, 331(4):042003, 2011.

[3] Pekka Enberg and Glauber Costa. The missing abstraction for AI agents: The agent filesystem. Turso engineering blog, November 2025. Published 2025-11-13; official software announcement, not a peer-reviewed paper.

[4] Tyler Harter, Brandon Salmon, Rose Liu, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Slacker: Fast distribution with lazy Docker containers. In *14th USENIX Conference on File and Storage Technologies (FAST 16)*, pages 181–195. USENIX Association, 2016.

[5] Cheng Huang, Huseyin Simitci, Yikang Xu, Aaron Ogus, Brad Calder, Parikshit Gopalan, Jin Li, and Sergey Yekhanin. Erasure coding in Windows Azure Storage. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)*, pages 15–26. USENIX Association, 2012.

[6] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024.

[7] Juicedata, Inc. JuiceFS architecture. JuiceFS Community Edition documentation. Official technical documentation; page last updated 2025-11-06, accessed 2026-10-22.

[8] Timothy Kim, Sanjith Athlur, Saurabh Kadekodi, Francisco Maturana, Dax Delvira, Arif Merchant, Gregory R. Ganger, and K. V. Rashmi. Morph: Efficient file-lifetime redundancy management for cluster file systems. In *Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP '24)*, pages 330–346. Association for Computing Machinery, 2024.

[9] Huiba Li, Yifan Yuan, Rui Du, Kai Ma, Lanzheng Liu, and Windsor Hsu. DADI: Block-level image service for agile and elastic application deployment. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 727–740. USENIX Association, 2020.

[10] Jiahao Li, Biao Cao, Jielong Jian, Cheng Li, Sen Han, Yiduo Wang, Yufei Wu, Kang Chen, Zhihui Yin, Qiushi Chen, Jiwei Xiong, Jie Zhao, Fengyuan Liu, Yan Xing, Liguo Duan, Miao Yu, Ran Zheng, Feng Wu, and Xianjun Meng. Mantle: Efficient hierarchical metadata management for cloud object storage services. In *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP '25)*, pages 928–943. Association for Computing Machinery, 2025.

[11] Siyang Li, Youyou Lu, Jiwu Shu, Yang Hu, and Tao Li. LocoFS: A loosely-coupled metadata service for distributed file systems. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '17)*, 2017.

[12] Linux Kernel Contributors. Overlay filesystem. Linux kernel documentation. Official technical documentation, accessed 2026-10-22; pin the tested kernel and mount options in the evaluation.

[13] Wenhao Lv, Youyou Lu, Yiming Zhang, Peile Duan, and Jiwu Shu. InfiniFS: An efficient metadata service for large-scale distributed filesystems. In *20th USENIX Conference on File and Storage Technologies (FAST 22)*, pages 313–328. USENIX Association, 2022.

[14] Nydus Authors. Registry accelerated file system. Nydus official documentation. Official technical documentation, accessed 2026-10-22; not a peer-reviewed paper.

[15] Satadru Pan, Theano Stavrinos, Yunqiao Zhang, Atul Sikaria, Pavel Zakharov, Abhinav Sharma, Shiva Shankar P, Mike Shuey, Richard Wareing, Monika Gangapuram, Guanglei Cao, Christian Preseau, Pratap Singh, Kestutis Patiejunas, JR Tipton, Ethan Katz-Bassett, and Wyatt Lloyd. Facebook's Tectonic filesystem: Efficiency from exascale. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pages 217–231. USENIX Association, 2021.

[16] Kai Ren and Garth Gibson. TABLEFS: Enhancing metadata efficiency in the local file system. In *2013 USENIX Annual Technical Conference (USENIX ATC 13)*, pages 145–156. USENIX Association, 2013.

[17] Kai Ren, Qing Zheng, Swapnil Patil, and Garth Gibson. IndexFS: Scaling file system metadata performance with stateless caching and bulk insertion. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '14)*, 2014.

[18] Cong Wang and Yusheng Zheng. Fork, explore, commit: OS primitives for agentic exploration. In *Agentic OS Workshop at ASPLOS 2026*, March 2026. Workshop paper; arXiv:2602.08199v2. The proposed branch() syscall is not implemented in the evaluated prototype.

[19] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Daniel Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025.

[20] Sage A. Weil, Scott A. Brandt, Ethan L. Miller, Darrell D. E. Long, and Carlos Maltzahn. Ceph: A scalable, high-performance distributed file system. In *7th USENIX Symposium on Operating Systems Design and Implementation (OSDI 06)*, pages 307–320. USENIX Association, 2006.

[21] Jingwei Xu, Mingkai Dong, Qiulin Tian, Ziyi Tian, Tong Xin, and Haibo Chen. SwitchFS: Asynchronous metadata updates for distributed filesystems with in-network coordination. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys '26)*. Association for Computing Machinery, 2026.

[22] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[23] Qing Zheng, Charles D. Cranor, Gregory R. Ganger, Garth A. Gibson, George Amvrosiadis, Bradley W. Settlemyer, and Gary Grider. DeltaFS: A scalable no-ground-truth filesystem for massively-parallel computing. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '21)*, 2021.

[24] Shawn Wanxiang Zhong, Junxuan Liao, Jing Liu, Mai Zheng, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Don't let AI agents YOLO your files: Information and control in agent-native filesystems. In *Proceedings of the 32nd ACM Symposium on Operating Systems Principles (SOSP '26)*. Association for Computing Machinery, 2026.